

มันฝรั่ง

- เวลาจำกัด: 10 นาที
- สภาพแวดล้อม: GPU 1 ตัว (VRAM ≈16 GB) ไม่มีอินเทอร์เน็ต
- ขนาดโซลูชัน: `solution.ipynb` ≤ 1 MB
- พื้นที่จัดเก็บ: 5 GB

โจทย์

เพื่อนของคุณชวนเล่นเกมทายคำ เขาซึ่งทำหน้าที่เป็นกรรมการจะเลือกคำลับหนึ่งคำจากคลังคำศัพท์ที่กำหนดไว้ และคุณต้องหาคำนั้นให้พบภายในไม่เกิน 30 รอบ ในแต่ละรอบกรรมการจะเปรียบเทียบคำสองคำและรายงานว่าคำใดมีความหมายใกล้เคียงกับ คำลับมากกว่า ทุกเกมเริ่มต้นจาก คู่คำที่กำหนดไว้ `lamp vs potato` เนื่องจากทั้งสองคำเป็นสองสิ่งที่เพื่อนของคุณชื่นชอบ จากนั้นโปรแกรมของคุณ จะเสนอคำใหม่หนึ่งคำ คำที่ชนะการเปรียบเทียบจะถูกเก็บไว้ และนำไปเปรียบเทียบกับคำที่คุณเสนอถัดไป คุณชนะเกมทันทีที่เสนอคำลับได้ตรงกับทุกประการ การจับคู่ ไม่คำนึงถึงตัวพิมพ์เล็กและตัวพิมพ์ใหญ่ ทุกคำที่คุณเสนอต้องอยู่ใน `dataset/vocabulary.json`

มีตัวอย่างฉบับสมบูรณ์ใน `solution.ipynb` ซึ่งประกอบด้วยไพธอนโค้ดและการโหลดข้อมูล คุณสามารถแก้ไขคลาส `PublicEmbeddingPlayer` ได้ โปรแกรมของคุณจะถูกเริ่มต้นหนึ่งครั้งและเล่นทุกเกมในการรันครั้งเดียว ไพธอนโค้ดจะสร้าง `PublicEmbeddingPlayer` ใหม่เมื่อเริ่มต้นแต่ละเกม

กรรมกร

โปรแกรมของคุณส่งออบเจกต์ JSON หนึ่งรายการไปยังกรรมการ และกรรมการจะตอบกลับด้วยออบเจกต์ JSON หนึ่งรายการ

ตัวอย่างการทำงาน โดยแสดงคำลับเพื่ออธิบายไพธอนโค้ดเท่านั้น:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}          <- matches: game won
```

รอบต่าง ๆ มีดัชนีตั้งแต่ 1 ถึง 30

ตัวเลือกของ `verdict` ได้แก่ `first` ซึ่งหมายความว่า `word1` ใกล้เคียงกว่า, `second` ซึ่งหมายความว่า `word2` ใกล้เคียงกว่า หรือ `same` ซึ่งหมายความว่าทั้งสองคำใกล้เคียงกับคำลับเท่ากัน

`winner_word` คือคำที่เก็บไว้สำหรับการเปรียบเทียบครั้งถัดไป เมื่อผลตัดสินเป็น `same` คำแรกจะยังคงอยู่

ชุดข้อมูล

ใช้ร่วมกันในทุก split:

- `dataset/vocabulary.json` — คำศัพท์ที่เลือกที่ไม่ซ้ำกัน 1602 คำ คำลับจะเป็น หนึ่งในคำเหล่านี้เสมอ
- `dataset/public_embeddings.npy` — `float32`, รูปร่าง (1602, 2560) แถว `i` สอดคล้องกับคำ `i` ในคลังคำศัพท์ คำเหล่านี้คือ `embedding` สาธารณะ

แต่ละ split คือเซตของคำลับ:

Split	คำ	คำตอบ	ใช้เพื่อ
<code>test_public</code>	120	 <code>dataset/test_public.json</code>	รับโซลูชันของคุณและคำนวณคะแนนด้วยตนเอง
<code>test_leaderboard_a</code>	120	ซ่อนอยู่	leaderboard แบบสด
<code>test_leaderboard_b</code>	120	ซ่อนอยู่	การจัดอันดับรอบสุดท้าย

ไม่มี `split train` — ไม่มีสิ่งใดถูก fit จากแถวที่มีป้ายกำกับ

โมเดลที่จัดเตรียมไว้

มีโมเดล embedding ที่ฝึกล่วงหน้าแล้วสองโมเดลมาพร้อมกับโจทย์และสามารถนำไปใช้ได้:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

ทั้งสองโมเดลต้องโหลดจากพารายในเครื่องของแต่ละโมเดล การใช้ Hugging Face hub id เช่น "BAAI/bge-m3" จะทำให้เกิดการดาวน์โหลดและล้มเหลว เนื่องจากการตัดสันทาแบบออฟไลน์ แต่ละ โดเรกทอรีมี example.py ที่รันได้ ซึ่งแสดงการเรียกใช้งานแบบออฟไลน์

ไลบรารีที่ใช้ได้: numpy, torch, sentence-transformers ไม่มีอินเทอร์เน็ท ไม่มี การดาวน์โหลด และไม่มีแพ็คเกจอื่น

เอาต์พุต

ไม่มี นี่เป็นโจทย์แบบโต้ตอบ: โซลูชันของคุณจะไม่เขียนไฟล์คำตอบ แต่จะสื่อสารกับ กระบวนการผ่าน stdin/stdout ตามที่อธิบายไว้ข้างต้น

เมตริก

เกมที่หาคำพบในรอบ t จะได้คะแนน $1.0 - 0.02 \times \max(0, t - 10)$ ส่วนเกมที่แก้ไม่สำเร็จ ภายใน 30 รอบจะได้คะแนน 0 ดังนั้นรอบ 1-10 จะได้คะแนน 1.00, รอบ 20 จะได้คะแนน 0.80 และรอบ 30 จะได้คะแนน 0.60

คะแนนโจทย์ของคุณคือคะแนนเฉลี่ยของเกม $\times 100$ ซึ่งอยู่ระหว่าง 0.00 และ 100.00

เวลาจำกัด 10 นาทีเป็นงบเวลาเดียวที่ครอบคลุมการเริ่มต้น การเตรียมการ และเกมทั้ง 120 เกมในชุดทดสอบ

วิธีส่ง

1. เปิด solution.ipynb แก้ไข PublicEmbeddingPlayer และรันทุกเซลล์เพื่อตรวจสอบว่าทำงานได้
2. หากต้องการ ให้ตรวจสอบภายในเครื่อง: `python local_test.py solution.ipynb --limit 5` กระบวนการภายในเครื่องใช้ embedding *สาธารณะ* ดังนั้นคะแนนที่ได้ จึงเป็นเพียงแนวทาง
3. บันทึก solution.ipynb
4. เปิดเก็บ Git ในแถบด้านซ้ายของ JupyterLab
5. stage solution.ipynb (ไอคอน + ที่อยู่ข้างไฟล์)
6. ป้อนข้อความ commit แล้วคลิก Commit
7. คลิกไอคอนรูปเมฆพร้อมลูกศรชี้ขึ้นเพื่อ push
8. กลับมายังหน้าการแข่งขันนี้แล้วคลิก Submit โดยข้อความ commit ต้องตรงกับข้อความที่คุณระบุไว้

ส่งไฟล์เพียงไฟล์เดียวชื่อ solution.ipynb ซึ่งครอบคลุมการเตรียมการและการอนุมานที่จำเป็นทั้งหมด